🤖 Machine Learning na dużych zbiorach danych

MLlib w Spark — gdy scikit-learn mówi "nie dam rady"! 🏋️💥

Python w Zastosowaniach Big Data | Wykład 5

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Recap scikit-learn — co umiemy i dlaczego to nie wystarczy
2. Dlaczego MLlib? — od jednej maszyny do klastra
3. MLlib Overview — architektura i kluczowe pakiety
4. Przygotowanie danych w MLlib (VectorAssembler, encodery, skalowanie)
5. ML Pipelines w Spark — fit/transform na sterydach
6. Modele klasyfikacji i regresji
7. Ewaluacja modelu — metryki w skali Big Data
8. Hyperparameter Tuning — CrossValidator i ParamGrid
9. Zapis i odczyt modeli
10. Praktyczny demo — end-to-end! 🚀

> 🎯 **Po tym wykładzie:** Będziesz potrafić zbudować i zewaluować model ML na terabajtach danych w Spark!

💡 Hook — Dlaczego tu jesteśmy?

Twój model scikit-learn działa pięknie na **100 000 wierszy**. Ale co zrobisz, gdy szef zleci Ci analizę **10 miliardów rekordów**? 🤯

```
# To działa super na laptopie...
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_train, y_train)  # 👈 100K wierszy — ok!
```

```
# ...ale to? 💀
model.fit(X_train_10_billion, y_train_10_billion)
# MemoryError: Unable to allocate 74.5 GiB 😱
```

> Czas poznać **MLlib** — Machine Learning w skali Big Data! 🔥

🔄 Recap: Co umiemy z scikit-learn?

Na Wykładzie 4 nauczyliśmy się:

Koncept	scikit-learn	Status
Preprocessing	`SimpleImputer`, `StandardScaler`	✅ Umiemy!
Encoding	`OneHotEncoder`, `LabelEncoder`	✅ Umiemy!
Pipeline	`sklearn.pipeline.Pipeline`	✅ Umiemy!
Modele	`LogisticRegression`, `RandomForest`	✅ Umiemy!
Ewaluacja	`accuracy_score`, `f1_score`, `r2_score`	✅ Umiemy!
Train/Test Split	`train_test_split()`	✅ Umiemy!

> 💪 Świetna baza! Teraz **skalujemy** to wszystko na klaster Spark!

⚠️ Problem: scikit-learn nie skaluje się

scikit-learn działa na **jednej maszynie** i wymaga, by dane zmieściły się w **RAM**:

Scenariusz	Rozmiar danych	scikit-learn	MLlib
Analiza ankiet studentów	10 MB	✅ Instant	⚡ Overkill
E-commerce sklep	1 GB	✅ 30s	✅ 1min
Logi serwera firmy	100 GB	💀 OOM	✅ 5min
Dane telekomunikacji	1 TB	🪦 Nieosiągalne	✅ 30min
Dane sensorów IoT	10 TB	🪦 Nieosiągalne	✅ 2h

> 🔑 **Zasada kciuka:** Dane < 10 GB → scikit-learn. Dane > 10 GB → **MLlib**!

🤔 Think About It

Pytanie: Wyobraźcie sobie, że pracujecie w firmie logistycznej. Macie dane GPS z 50 000 ciężarówek, zbierane co sekundę, przez 5 lat.

- Ile to **rekordów**? 🤔
- Czy zmieszczą się w RAM waszego laptopa?

> To właśnie scenariusz, w którym MLlib ratuje sytuację! 🦸

🔥 Dlaczego MLlib?

MLlib (Machine Learning Library) — biblioteka ML wbudowana w Apache Spark.

Cecha	scikit-learn	MLlib
Wykonanie	Jednowątkowe / lokalne	**Rozproszone** na klastrze
Limit danych	RAM jednej maszyny	**Brak limitu** (klaster)
Obliczenia	CPU jednej maszyny	**Setki/tysiące** rdzeni
Integracja	pandas, NumPy	**Spark DataFrames**
Deployment	Flask/FastAPI	**Spark Serving / MLflow**

> 💡 MLlib to jak scikit-learn, ale rozłożony na **wiele maszyn jednocześnie**!

MLlib — Architektura

```
spark.ml (DataFrame-based API) ← ✅ Używamy tego!
    ├── feature     → VectorAssembler, StringIndexer, Scaler...
    ├── classification → LogisticRegression, RandomForest...
    ├── regression   → LinearRegression, GBTRegressor...
    ├── clustering   → KMeans, BisectingKMeans...
    ├── evaluation   → BinaryClassificationEvaluator...
    ├── tuning       → CrossValidator, ParamGridBuilder...
    └── pipeline     → Pipeline, PipelineModel

spark.mllib (RDD-based API) ← ❌ Stare! Nie używamy!
```

> ⚠️ **Uwaga:** Istnieją **dwa** API — `spark.ml` (nowe, DataFrame) i `spark.mllib` (stare, RDD). My zawsze używamy `spark.ml`!

🚀 Start — SparkSession

Zanim zaczniemy, potrzebujemy SparkSession:

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
         .appName("MLlib-Wyklad5")
         .master("local[*]")      # 👈 Lokalnie: wszystkie rdzenie
         .getOrCreate())

# Wczytujemy dane
df = (spark.read
      .option("header", True)
      .option("inferSchema", True)
      .csv("customers.csv"))

df.printSchema()
df.show(5)
```

> 💡 Na klastrze zamienimy `local[*]` na adres cluster managera — kod reszty **nie zmieni się**!

📊 Przykładowe dane — dataset klientów

```
# Dane klientów banku – przewidujemy czy klient odejdzie (churn)
df.show(5, truncate=False)
```

```
+---+------+---+---------+--------+------+-----+
|id |gender|age|tenure   |balance |salary |churn|
+---+------+---+---------+--------+------+-----+
|1  |Male  |42 |2        |0.0     |101349 |1    |
|2  |Female|41 |1        |83808.0 |112543 |0    |
|3  |Female|42 |8        |159661.0|113932 |1    |
|4  |Male  |39 |1        |0.0     |93827  |0    |
|5  |Female|43 |2        |125511.0|79084  |0    |
+---+------+---+---------+--------+------+-----+
```

> 🎯 Cel: przewidzieć kolumnę **churn** (0 = zostaje, 1 = odchodzi)

⚙️ VectorAssembler — Krok #1

W scikit-learn podajemy features jako **macierz NumPy** / pandas DataFrame.
W MLlib **wszystkie cechy** muszą być w **jednej kolumnie** typu `Vector`:

```
from pyspark.ml.feature import VectorAssembler

# scikit-learn:      X = df[["age", "tenure", "balance", "salary"]]
# MLlib:             ↓↓↓

assembler = VectorAssembler(
    inputCols=["age", "tenure", "balance", "salary"],  # 👈 Lista cech
    outputCol="features"                                 # 👈 Nowa kolumna
)

df_assembled = assembler.transform(df)
df_assembled.select("features", "churn").show(3, truncate=False)
# +-----------------------------+-----+
# |features                     |churn|
# +-----------------------------+-----+
# |[42.0,2.0,0.0,101349.0]      |1    |
# |[41.0,1.0,83808.0,112543.0]  |0    |
# +-----------------------------+-----+
```

🔤 StringIndexer — Encoding kategorii

StringIndexer zamienia kolumny tekstowe na **indeksy numeryczne** (jak `LabelEncoder` w sklearn):

```
from pyspark.ml.feature import StringIndexer

indexer = StringIndexer(
    inputCol="gender",       # 👈 Kolumna tekstowa
    outputCol="gender_idx"   # 👈 Nowa kolumna numeryczna
)

df_indexed = indexer.fit(df).transform(df)
df_indexed.select("gender", "gender_idx").show(4)
# +------+----------+
# |gender|gender_idx|
# +------+----------+
# |Male  |0.0       |   👈 Najczęstsza kategoria = 0
# |Female|1.0       |
# |Female|1.0       |
# |Male  |0.0       |
# +------+----------+
```

> 💡 StringIndexer sortuje kategorie wg **częstości** — najczęstsza = 0, rzadsza = 1, itd.

🔥 OneHotEncoder — Rozbijanie kategorii

Po `StringIndexer` możemy zastosować **OneHotEncoder** (jak w sklearn):

```
from pyspark.ml.feature import OneHotEncoder

encoder = OneHotEncoder(
    inputCol="gender_idx",         # 👈 Wejście z StringIndexer
    outputCol="gender_encoded"     # 👈 Wyjście: SparseVector
)

df_encoded = encoder.fit(df_indexed).transform(df_indexed)
df_encoded.select("gender", "gender_idx", "gender_encoded").show(4)
# +------+----------+--------------+
# |gender|gender_idx|gender_encoded|
# +------+----------+--------------+
# |Male  |0.0       |(1,[0],[1.0]) |   👈 SparseVector!
# |Female|1.0       |(1,[],[])     |
# +------+----------+--------------+
```

> ⚠️ W MLlib `OneHotEncoder` domyślnie pomija **ostatnią** kategorię (drop last) — to **poprawne** zachowanie!

📏 StandardScaler — Skalowanie cech

Niektóre algorytmy (LogisticRegression, SVM) wymagają **znormalizowanych** danych:

```
from pyspark.ml.feature import StandardScaler

scaler = StandardScaler(
    inputCol="features",          # 👈 Kolumna z VectorAssembler
    outputCol="scaled_features",  # 👈 Nowa, przeskalowana
    withStd=True,                 # 👈 Dziel przez odchylenie std
    withMean=False                # 👈 Sparse vectors: mean=False!
)

scaler_model = scaler.fit(df_assembled)
df_scaled = scaler_model.transform(df_assembled)
df_scaled.select("features", "scaled_features").show(2, truncate=False)
```

> ⚠️ Dla **Sparse Vectors** ustawiamy `withMean=False` — inaczej MLlib wyrzuci błąd!

🩹 Imputer — Brakujące wartości

```
from pyspark.ml.feature import Imputer

imputer = Imputer(
    inputCols=["age", "balance", "salary"],
    outputCols=["age_imp", "balance_imp", "salary_imp"],
    strategy="median"  # 👈 "mean", "median" lub "mode"
)

df_imputed = imputer.fit(df).transform(df)
df_imputed.select("age", "age_imp", "balance", "balance_imp").show(3)
```

Porównanie z scikit-learn:

scikit-learn	MLlib
`SimpleImputer(strategy="median")`	`Imputer(strategy="median")`
`fit_transform(X)`	`imputer.fit(df).transform(df)`
Obsługuje NaN/None	Obsługuje NaN (nie None!)

> 💡 W MLlib `Imputer` zamienia NaN, ale **nie** None — najpierw konwertuj None → NaN!

🔑 Kluczowa różnica: fit() + transform()

Wzorzec jest **identyczny** jak w scikit-learn:

```
# scikit-learn
scaler = StandardScaler()
scaler.fit(X_train)              # 👈 Ucz się parametrów
X_train = scaler.transform(X_train)  # Transformuj

# MLlib — ten sam wzorzec!
scaler = StandardScaler(inputCol="features", outputCol="scaled")
scaler_model = scaler.fit(df_train)        # 👈 Ucz się parametrów
df_train = scaler_model.transform(df_train)  # Transformuj
```

> 🔑 **Różnica:** W MLlib `fit()` zwraca **Model** (oddzielny obiekt), a `transform()` woła się na Modelu.

🏗️ ML Pipeline — Łączymy wszystko!

Ręczne woływanie fit/transform na każdym kroku jest **upierdliwe**. Pipeline ratuje!

```
from pyspark.ml import Pipeline

pipeline = Pipeline(stages=[
    StringIndexer(inputCol="gender", outputCol="gender_idx"),
    OneHotEncoder(inputCol="gender_idx", outputCol="gender_enc"),
    VectorAssembler(
        inputCols=["age", "tenure", "balance", "salary", "gender_enc"],
        outputCol="features"
    ),
    StandardScaler(inputCol="features", outputCol="scaled_features"),
    LogisticRegression(
        featuresCol="scaled_features",  # 👈 Cechy
        labelCol="churn"                # 👈 Target
    )
])
```

> 💡 Jeden obiekt, wiele kroków — dokładnie jak `sklearn.pipeline.Pipeline`!

🏗️ Pipeline — fit() i transform()

```
# Dzielimy dane na train/test
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)

# Trenujemy cały pipeline jednym wywołaniem!
model = pipeline.fit(train_df)      # 👈 Fit na train

# Predykcja na test
predictions = model.transform(test_df)  # 👈 Transform na test

predictions.select("churn", "prediction", "probability").show(5)
# +-----+----------+--------------------+
# |churn|prediction|probability         |
# +-----+----------+--------------------+
# |0    |0.0       |[0.82, 0.18]        |
# |1    |1.0       |[0.31, 0.69]        |
# +-----+----------+--------------------+
```

> 🎯 Pipeline automatycznie wywołuje `fit()` na Estimatorach i `transform()` na Transformerach!

🔄 Pipeline: scikit-learn vs MLlib

```
# scikit-learn
from sklearn.pipeline import Pipeline as SkPipeline
sk_pipe = SkPipeline([
    ("scaler", StandardScaler()),     # 👈 Krotka (nazwa, obiekt)
    ("model", LogisticRegression())
])
sk_pipe.fit(X_train, y_train)

# MLlib
from pyspark.ml import Pipeline
ml_pipe = Pipeline(stages=[
    StandardScaler(inputCol="f", outputCol="sf"),
    LogisticRegression(featuresCol="sf", labelCol="label")
])
ml_pipe.fit(train_df)             # 👈 Lista stages
```

Aspekt	scikit-learn	MLlib
Stages	Lista krotek `(name, obj)`	Lista obiektów `stages=[]`
Fit	`pipe.fit(X, y)`	`pipe.fit(df)`
Target	Osobny argument `y`	Kolumna w DataFrame (`labelCol`)

📊 Modele klasyfikacji w MLlib

Model	Klasa MLlib	Opis
Regresja logistyczna	`LogisticRegression`	Klasyka — szybka, interpretowalna
Drzewo decyzyjne	`DecisionTreeClassifier`	Proste, łatwe do wizualizacji
Random Forest	`RandomForestClassifier`	Ensemble — lepszy niż jedno drzewo
GBT	`GBTClassifier`	Gradient Boosted Trees — TOP! 🏆

```
from pyspark.ml.classification import (
    LogisticRegression,
    DecisionTreeClassifier,
    RandomForestClassifier,
    GBTClassifier
)
```

> 🔑 GBTClassifier to odpowiednik **XGBoost** / **LightGBM** w świecie Spark!

🌲 RandomForestClassifier — Przykład

```
from pyspark.ml.classification import RandomForestClassifier

rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="churn",
    numTrees=100,          # 👈 Liczba drzew w lesie
    maxDepth=5,            # 👈 Max głębokość drzewa
    seed=42
)

# W Pipeline:
pipeline_rf = Pipeline(stages=[
    assembler,   # VectorAssembler
    rf           # RandomForestClassifier
])

model_rf = pipeline_rf.fit(train_df)
predictions_rf = model_rf.transform(test_df)

# Feature importance — które cechy są ważne?
print(model_rf.stages[-1].featureImportances)
# (4,[0,1,2,3],[0.35,0.12,0.28,0.25])
# 👈 age=35%, tenure=12%, balance=28%, salary=25%
```

⚡ GBTClassifier — Gradient Boosted Trees

```
from pyspark.ml.classification import GBTClassifier

gbt = GBTClassifier(
    featuresCol="features",
    labelCol="churn",
    maxIter=50,          # 👈 Liczba iteracji boostingu
    maxDepth=5,          # 👈 Głębokość drzew
    stepSize=0.1,        # 👈 Learning rate
    seed=42
)

pipeline_gbt = Pipeline(stages=[assembler, gbt])
model_gbt = pipeline_gbt.fit(train_df)
predictions_gbt = model_gbt.transform(test_df)
```

> 💡 **GBT** buduje drzewa **sekwencyjnie** — każde następne poprawia błędy poprzedniego.
> To jak poprawianie egzaminu: za każdym razem skupiasz się na pytaniach, które wcześniej źle zrobiłeś! 📝

📈 Modele regresji w MLlib

Gdy cel jest **liczbą ciągłą** (cena, temperatura, przychód):

```
from pyspark.ml.regression import (
    LinearRegression,
    RandomForestRegressor,
    GBTRegressor
)

# Przykład: przewidujemy saldo klienta
lr = LinearRegression(
    featuresCol="features",
    labelCol="balance",
    maxIter=100,
    regParam=0.01,     # 👈 Regularyzacja L2
    elasticNetParam=0  # 👈 0=L2 (Ridge), 1=L1 (Lasso)
)

pipeline_reg = Pipeline(stages=[assembler, lr])
model_reg = pipeline_reg.fit(train_df)

# Podsumowanie modelu
summary = model_reg.stages[-1].summary
print(f"RMSE: {summary.rootMeanSquaredError:.2f}")
print(f"R²:   {summary.r2:.4f}")
```

🤔 Think About It — Który model wybrać?

Sytuacja	Zalecany model	Dlaczego?
Szybki baseline	`LogisticRegression`	Prosta, szybka, interpretowalność
Nieliniowe zależności	`RandomForestClassifier`	Łapie złożone wzorce
Maksymalna dokładność	`GBTClassifier`	Najlepsza jakość (ale wolniejszy)
Dużo kategorii/cech	`RandomForest`	Odporny na overfitting
Potrzebujesz wyjaśnień	`DecisionTreeClassifier`	Łatwy do wizualizacji

> 🎯 **Protip:** Zacznij od `LogisticRegression` jako **baseline**, potem próbuj `RandomForest` i `GBT`!

📊 Ewaluacja modelu — BinaryClassificationEvaluator

Dla problemów **klasyfikacji binarnej** (tak/nie, churn/nie-churn):

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator

evaluator = BinaryClassificationEvaluator(
    labelCol="churn",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"     # 👈 AUC-ROC!
)

auc = evaluator.evaluate(predictions)
print(f"AUC-ROC: {auc:.4f}")  # AUC-ROC: 0.8543

# Można też policzyć AUC-PR (Area Under Precision-Recall)
evaluator_pr = BinaryClassificationEvaluator(
    labelCol="churn",
    metricName="areaUnderPR"      # 👈 Lepsze przy niezbalansowanych
)
auc_pr = evaluator_pr.evaluate(predictions)
print(f"AUC-PR: {auc_pr:.4f}")
```

> 💡 **AUC-ROC** = 0.5 → model losowy 🎲, AUC-ROC = 1.0 → model idealny 🏆

📊 MulticlassClassificationEvaluator

Dla problemów z **wieloma klasami** (A/B/C/D):

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

evaluator_mc = MulticlassClassificationEvaluator(
    labelCol="churn",
    predictionCol="prediction"
)

for metric in ["accuracy", "f1", "weightedPrecision"]:
    evaluator_mc.setMetricName(metric)
    print(f"{metric}: {evaluator_mc.evaluate(predictions):.4f}")
```

> ⚠️ Pamiętajcie z Wykładu 4: **accuracy** kłamie przy niezbalansowanych klasach! Używajcie **F1**!

RegressionEvaluator

Dla problemów **regresji** (przewidywanie liczby):

```
from pyspark.ml.evaluation import RegressionEvaluator

evaluator_reg = RegressionEvaluator(
    labelCol="balance",
    predictionCol="prediction"
)

# RMSE — Root Mean Squared Error
evaluator_reg.setMetricName("rmse")
rmse = evaluator_reg.evaluate(pred_reg)

# MAE — Mean Absolute Error
evaluator_reg.setMetricName("mae")
mae = evaluator_reg.evaluate(pred_reg)

# R² — współczynnik determinacji
evaluator_reg.setMetricName("r2")
r2 = evaluator_reg.evaluate(pred_reg)

print(f"RMSE: {rmse:.2f}")   # Im niższe, tym lepiej
print(f"MAE:  {mae:.2f}")    # Średni błąd absolutny
print(f"R²:   {r2:.4f}")     # Im bliżej 1.0, tym lepiej
```

🔄 Porównanie metryk: scikit-learn vs MLlib

Metryka	scikit-learn	MLlib
AUC-ROC	`roc_auc_score(y, proba)`	`BinaryClassificationEvaluator(metricName="areaUnderROC")`
Accuracy	`accuracy_score(y, pred)`	`MulticlassClassificationEvaluator(metricName="accuracy")`
F1	`f1_score(y, pred)`	`MulticlassClassificationEvaluator(metricName="f1")`
RMSE	`np.sqrt(mean_squared_error(y, pred))`	`RegressionEvaluator(metricName="rmse")`
MAE	`mean_absolute_error(y, pred)`	`RegressionEvaluator(metricName="mae")`
R^2	`r2_score(y, pred)`	`RegressionEvaluator(metricName="r2")`

> 💡 Metryki **te same** — zmienia się tylko **API**, nie matematyka!

🎛️ Hyperparameter Tuning — ParamGridBuilder

Szukamy najlepszej kombinacji hiperparametrów:

```
from pyspark.ml.tuning import ParamGridBuilder

rf = RandomForestClassifier(
    featuresCol="features", labelCol="churn"
)

param_grid = (ParamGridBuilder()
    .addGrid(rf.numTrees, [50, 100, 200])     # 👈 3 wartości
    .addGrid(rf.maxDepth, [3, 5, 10])         # 👈 3 wartości
    .addGrid(rf.minInstancesPerNode, [1, 5])  # 👈 2 wartości
    .build())

print(f"Liczba kombinacji: {len(param_grid)}")
# Liczba kombinacji: 18  (3 × 3 × 2)
```

> ⚠️ Uważajcie na **eksplozję kombinacji**! 5 parametrów × 5 wartości = 3 125 kombinacji 😱

CrossValidator — Walidacja krzyżowa

```
from pyspark.ml.tuning import CrossValidator

cv = CrossValidator(
    estimator=pipeline_rf,           # 👈 Nasz Pipeline
    estimatorParamMaps=param_grid,   # 👈 Grid parametrów
    evaluator=BinaryClassificationEvaluator(
        labelCol="churn",
        metricName="areaUnderROC"
    ),
    numFolds=3,                      # 👈 3-fold CV
    parallelism=4,                   # 👈 Równoległe joby!
    seed=42
)

cv_model = cv.fit(train_df)  # 👈 Trenuje 18 × 3 = 54 modele!

# Najlepszy model
best_model = cv_model.bestModel
print(f"Best AUC: {max(cv_model.avgMetrics):.4f}")
```

> ⚡ Parametr `parallelism` pozwala trenować kilka modeli **jednocześnie** — mega przyspieszenie!

⚡ TrainValidationSplit — Szybsza alternatywa

Gdy dane są **ogromne** i CrossValidator trwa za długo:

```
from pyspark.ml.tuning import TrainValidationSplit

tvs = TrainValidationSplit(
    estimator=pipeline_rf,
    estimatorParamMaps=param_grid,
    evaluator=BinaryClassificationEvaluator(
        labelCol="churn",
        metricName="areaUnderROC"
    ),
    trainRatio=0.8,        # 👈 80% train, 20% validation
    parallelism=4,
    seed=42
)

tvs_model = tvs.fit(train_df)  # 👈 18 modeli (bez foldów!)
```

Metoda	Modele trenowane	Kiedy używać?
`CrossValidator` (k=3)	k × grid_size = **54**	Mniejsze dane, dokładniejsze
`TrainValidationSplit`	1 × grid_size = **18**	Duże dane, szybsze

💾 Zapis i odczyt modeli

```
# Zapis modelu na dysk (lub HDFS / S3 / GCS)
best_model.write().overwrite().save("models/churn_rf_v1")

# Odczyt modelu
from pyspark.ml import PipelineModel
loaded_model = PipelineModel.load("models/churn_rf_v1")

# Użycie załadowanego modelu
new_predictions = loaded_model.transform(new_data_df)
new_predictions.select("prediction", "probability").show(3)
```

> 💡 Model zapisuje się jako **folder** z metadanymi i wagami. Można go wrzucić na HDFS, S3 lub GCS!

📦 MLflow — Śledzenie eksperymentów (bonus)

MLflow to open-source platforma do zarządzania cyklem życia ML:

```
import mlflow
import mlflow.spark

# Logowanie eksperymentu
with mlflow.start_run(run_name="churn_rf_v1"):
    model = pipeline_rf.fit(train_df)
    predictions = model.transform(test_df)
    auc = evaluator.evaluate(predictions)

    mlflow.log_param("numTrees", 100)       # 👈 Parametry
    mlflow.log_param("maxDepth", 5)
    mlflow.log_metric("auc_roc", auc)       # 👈 Metryki
    mlflow.spark.log_model(model, "model")  # 👈 Model
```

> 🎯 MLflow pozwala **porównywać** десятки eksperymentów w dashboardzie. Nie musisz pamiętać, który model był najlepszy!

🔄 scikit-learn vs MLlib — Wielkie porównanie

Aspekt	scikit-learn 🐍	MLlib 🔥
Import	`from sklearn...`	`from pyspark.ml...`
Dane wejściowe	NumPy array / pandas	Spark DataFrame
Features	Macierz kolumn	Jedna kolumna `Vector`
Pipeline	`Pipeline([(name, obj)])`	`Pipeline(stages=[obj])`
fit()	`model.fit(X, y)`	`model.fit(df)`
predict()	`model.predict(X)`	`model.transform(df)`
CV	`cross_val_score()`	`CrossValidator`
Tuning	`GridSearchCV`	`CrossValidator + ParamGridBuilder`
Skalowanie	Jedna maszyna	Klaster (setki maszyn)
Speed (small data)	⚡ Szybszy	🐌 Wolniejszy (overhead)
Speed (big data)	💀 Niemożliwe	⚡ Dystrybuowane

🤔 Think About It — Kiedy co wybrać?

Scenariusz 1: Masz 50 000 rekordów i raporcik do szefa na jutro.
→ ✅ **scikit-learn** — szybciej, prostsze, wystarczające

Scenariusz 2: Masz 500 milionów rekordów logów serwera.
→ ✅ **MLlib** — jedyny rozsądny wybór

Scenariusz 3: Masz 5 milionów rekordów.
→ 🤔 **Zależy** — jeśli mieści się w RAM, sklearn. Jeśli rośnie, MLlib zapewnia skalowalność.

> 🔑 **Złota zasada:** Nie używaj armaty do zabicia muchy. Ale gdy mucha waży 10 TB — bierz armatę! 🎯

🚀 Praktyczny demo — End-to-End Pipeline

Zbudujmy kompletny model churn prediction:

```
from pyspark.sql import SparkSession
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    StringIndexer, OneHotEncoder,
    VectorAssembler, StandardScaler
)
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator

spark = (SparkSession.builder
         .appName("ChurnDemo")
         .master("local[*]")
         .getOrCreate())
```

> 🎬 Krok 1: Uruchamiamy Spark i importujemy wszystko, czego potrzebujemy!

🚀 Demo — Krok 2: Dane i Pipeline

```
df = (spark.read
      .option("header", True)
      .option("inferSchema", True)
      .csv("bank_customers.csv"))

# Budujemy Pipeline
pipeline = Pipeline(stages=[
    StringIndexer(inputCol="gender", outputCol="gender_idx"),
    OneHotEncoder(inputCol="gender_idx", outputCol="gender_enc"),
    VectorAssembler(
        inputCols=["age", "tenure", "balance", "salary", "gender_enc"],
        outputCol="features"
    ),
    StandardScaler(inputCol="features", outputCol="scaled"),
    RandomForestClassifier(
        featuresCol="scaled", labelCol="churn",
        numTrees=100, maxDepth=5, seed=42
    )
])
```

> 🎬 Krok 2: Cały preprocessing + model w **jednym** Pipeline!

🚀 Demo — Krok 3: Trening i ewaluacja

```
# Split
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)
print(f"Train: {train_df.count()}, Test: {test_df.count()}")

# Trenujemy
model = pipeline.fit(train_df)

# Predykcja
predictions = model.transform(test_df)

# Ewaluacja
evaluator = BinaryClassificationEvaluator(
    labelCol="churn", metricName="areaUnderROC"
)
auc = evaluator.evaluate(predictions)
print(f"🎯 AUC-ROC: {auc:.4f}")

# Confusion-like breakdown
predictions.groupBy("churn", "prediction").count().show()
```

🚀 Demo — Krok 4: Tuning

```
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

rf = pipeline.getStages()[-1]  # 👈 Ostatni stage = RF

param_grid = (ParamGridBuilder()
    .addGrid(rf.numTrees, [50, 100, 200])
    .addGrid(rf.maxDepth, [3, 5, 10])
    .build())

cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=param_grid,
    evaluator=evaluator,
    numFolds=3,
    parallelism=4,
    seed=42
)

cv_model = cv.fit(train_df)
best_auc = evaluator.evaluate(cv_model.transform(test_df))
print(f"🏆 Best AUC-ROC po tuning: {best_auc:.4f}")
```

🚀 Demo — Krok 5: Zapis i podsumowanie

```
# Zapisujemy najlepszy model
cv_model.bestModel.write().overwrite().save("models/churn_best")

# Podsumowanie wyniku
print("=" * 50)
print("📊 PODSUMOWANIE MODELU CHURN")
print("=" * 50)
print(f"Algorytm:   RandomForestClassifier")
print(f"AUC-ROC:    {best_auc:.4f}")
print(f"Folds CV:   3")
print(f"Grid size:  {len(param_grid)} kombinacji")
print(f"Model path: models/churn_best")
print("=" * 50)

# Kończymy sesję
spark.stop()
```

> 🎉 To cały proces! Od surowych danych CSV do **ztunowanego, zapisanego modelu** — w kilkudziesięciu linijkach!

⚠️ Najczęstsze pułapki — MLlib Edition

#	Pułapka 🕳️	Jak unikać ✅
1	Używanie `spark.mllib` (stare RDD API)	Zawsze `spark.ml` (DataFrame API)
2	Zapomnienie o `VectorAssembler`	Cechy muszą być w **jednej kolumnie** `Vector`
3	`withMean=True` na sparse vectors	`StandardScaler(withMean=False)`
4	Brak `inferSchema=True` przy odczycie CSV	Wszystkie kolumny będą stringami!
5	Cache'owanie danych po splicie	`train_df.cache()` — **ogromne** przyspieszenie
6	Za duży ParamGrid	Zacznij od mało kombinacji, rozszerzaj
7	Ewaluacja na train zamiast test	**Zawsze** testuj na niewidzianych danych!

> 🔑 Większość błędów wynika z różnic między scikit-learn a MLlib API — przyzwyczaicie się szybko!

📝 Kluczowe wnioski

1. ✅ **MLlib** to odpowiedź na Machine Learning w skali Big Data — gdy dane nie mieszczą się w RAM
2. ✅ `spark.ml` (DataFrame API) — jedyne właściwe API; `spark.mllib` jest deprecated
3. ✅ **VectorAssembler** — obowiązkowy krok; MLlib wymaga jednej kolumny features
4. ✅ **Pipeline** w MLlib działa jak w scikit-learn — `fit()` → `transform()`
5. ✅ **CrossValidator** + **ParamGridBuilder** = odpowiednik `GridSearchCV` ze sklearn
6. ✅ Metryki ewaluacji są **te same** co w scikit-learn — AUC, F1, RMSE, R^2

> 🧠 Zapamiętajcie: jeśli znacie scikit-learn — **znacie MLlib**. API jest inne, koncepty identyczne!

🔮 Co dalej?

Następny wykład: Streaming danych i przetwarzanie w czasie rzeczywistym (Spark Structured Streaming)

Ćwiczenia:

- 🏋️ Notebook: end-to-end ML Pipeline w PySpark na prawdziwym datasecie
- 🎯 Zadanie domowe: porównanie RandomForest vs GBT na swoich danych

Do poczytania:

- 📖 *Learning Spark, 2nd Edition* — Jules Damji et al. (rozdział o MLlib)
- 🔗 spark.apache.org/docs/latest/ml-guide.html — oficjalna dokumentacja
- 📝 Kaggle — dowolny dataset do przetestowania Pipeline w PySpark

❓ Pytania?

Dzięki za uwagę! 🎉

Podsumowanie w jednym zdaniu:

> scikit-learn + klaster Spark = **MLlib** — te same koncepty, ale w skali terabajtów!